Route Constraints in ASP.NET Core Web API

Route Constraints in ASP.NET Core Web API

In this article, I will discuss **Route Constraints in ASP.NET Core Web API** Applications with Examples. Please read our previous article discussing **Route Prefix in ASP.NET Core Web API** Routing.

In ASP.NET Core Web API, **Route Constraints** are validation rules applied to route parameters. They ensure that the incoming URL segments conform to specific **Data Types**, **Ranges**, or **Patterns** before the request reaches the controller action. This acts as an **Early Validation Layer**, preventing Invalid or Unexpected inputs from being processed by your action methods. This provides:

- Early Validation: Validate parameters *at the* routing level (before controller execution).
- Clear Route Disambiguation: Avoid ambiguity between similar routes.
- Improve Performance, Readability, and Security

They are written inside curly braces {} after the parameter name, using a colon : followed by the constraint name. The general syntax is: **{parameter:constraint}**

Commonly Used Route Constraints in ASP.NET Core Web API

The Commonly Used Route Constraints in ASP.NET Core Web API applications are as follows:

Type Constraint:

The type constraint restricts a route parameter to a specific data type, such as integer, double, bool, float, or datetime. It ensures that the incoming value matches the expected format; otherwise, the route won't be matched. For example, **{id:int}** allows only numeric IDs and rejects any non-numeric input like strings.
Syntax: {parameterName:type}

Min Constraint:

The min constraint defines the lowest permissible numeric value for a parameter. It ensures that the supplied number is equal to or greater than the specified minimum limit, preventing unrealistic or invalid inputs in numeric routes.

Syntax: {parameterName:int:min(value)}

Max Constraint:

The max constraint sets the highest permissible numeric value for a parameter. It ensures that the provided number does not exceed the defined maximum limit, commonly used in cases like price filters, pagination, or quantity validation.

Syntax: {parameterName:int:max(value)}

Range Constraint:

The range constraint specifies both minimum and maximum numeric limits for a route parameter. It validates that the value falls within the defined range (inclusive of both limits) before the action method executes.

Syntax: {parameterName:int:range(min,max)}

Alpha Constraint:

The alpha constraint allows only alphabetic characters (A–Z and a–z) for a route parameter. It is useful for parameters like names, categories, or regions where only letters are valid inputs.

Syntax: {parameterName:alpha}

MinLength Constraint:

The minlength constraint ensures that a string parameter contains at least the specified number of characters. It is often used to validate codes, usernames, or tokens that must meet a minimum length requirement.

Syntax: {parameterName:minlength(number)}

MaxLength Constraint:

The maxlength constraint ensures that a string parameter does not exceed the defined number of characters. It helps prevent overly long inputs in cases like product codes, coupon codes, or reference numbers.

Syntax: {parameterName:maxlength(number)}

Length Constraint:

The length constraint specifies that a string parameter must have an exact number of characters. It is ideal for fixed-format identifiers like ticket numbers, serial codes, or account IDs.

Syntax: {parameterName:length(number)}

Regex Constraint:

The regex constraint uses a regular expression to define a custom pattern that the route parameter must match. It provides the highest flexibility and is suitable for structured formats such as SKU codes, invoice numbers, or postal codes.

Syntax: {parameterName:regex(expression)}

Example Without Route Constraints: Ambiguous Route Issue

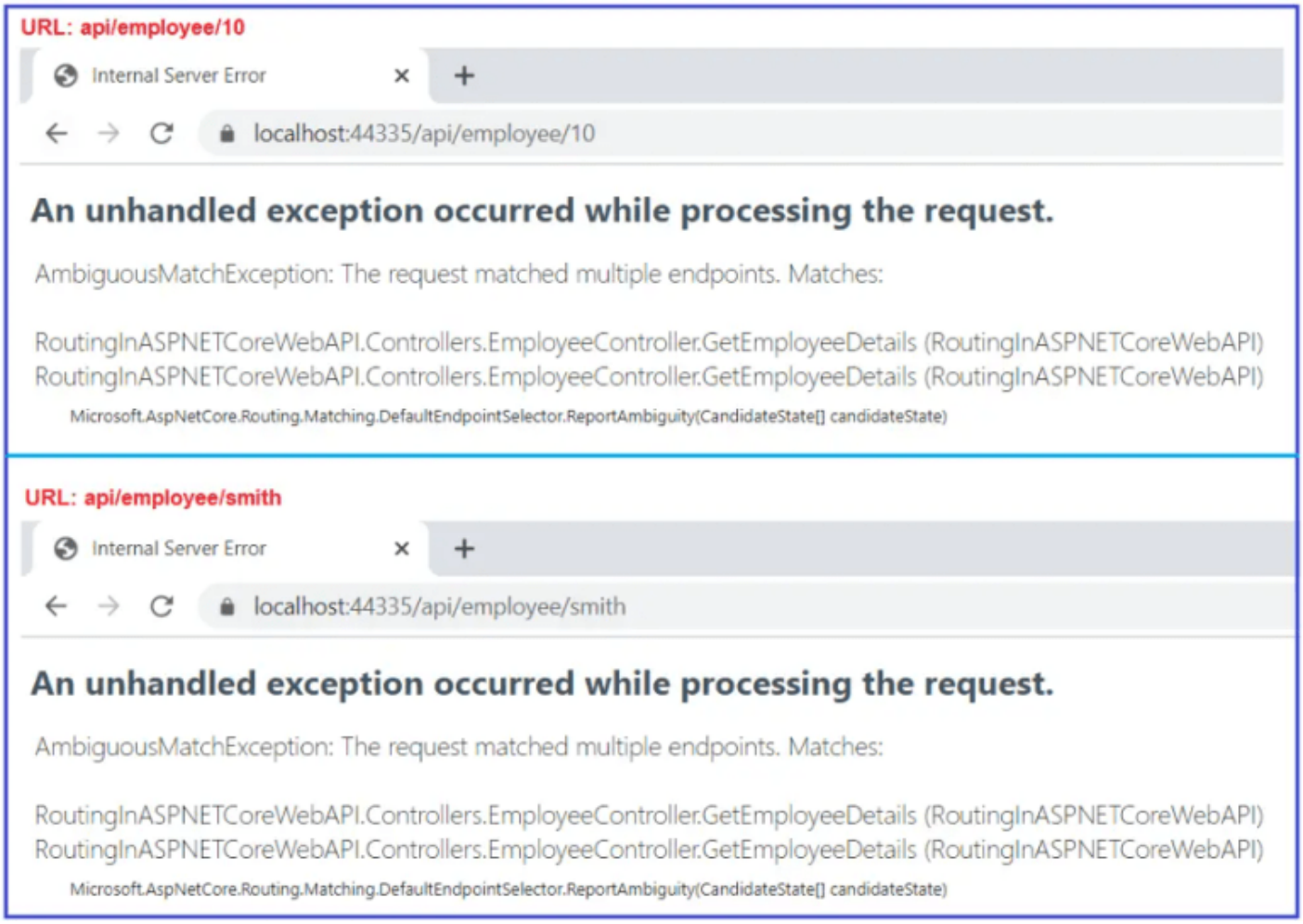
Let's begin with a situation **Without Constraints** to understand why they're essential. Without constraints, ASP.NET Core may not know which action to call when multiple routes match the same pattern. Please modify the Employee Controller class as shown below.

```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeeController : ControllerBase
    {
        [Route("{EmployeeId}")]
        [HttpGet]
        public string GetEmployeeDetails(int EmployeeId)
        {
            return $"Response from GetEmployeeDetails Method, EmployeeId :
{EmployeeId}";
        }

        [Route("{EmployeeName}")]
        [HttpGet]
        public string GetEmployeeDetails(string EmployeeName)
        {
            return $"Response from GetEmployeeDetails Method, EmployeeName :
{EmployeeName}";
        }
    }
}
```

Problem

If you navigate to **/api/employee/10** or **/api/employee/smith**, both routes match, leading to an **AmbiguousMatchException**, since the routing engine can't decide which method to invoke.



In real-world APIs, this occurs when identifiers such as EmployeeId and EmployeeName are both part of the URL. Constraints help the routing engine decide which action should handle the request. This is where **Route Constraints** come into play. They allow the framework to distinguish routes based on the parameter type or pattern.

Solution using int constraint

The int constraint ensures that the parameter accepts only integer values. If a user tries /api/employee/abc, the route will **not** match, avoiding type-conversion errors. We don't need to specify anything for the string parameter, as, by default, all the parameters in the ASP.NET Core are string only. Let's modify the Employee Controller to use the int-type Route Constraints as shown below.

```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeeController : ControllerBase
    {
        [Route("{EmployeeId:int}")]
        [HttpGet]
        public string GetEmployeeDetails(int EmployeeId)
```

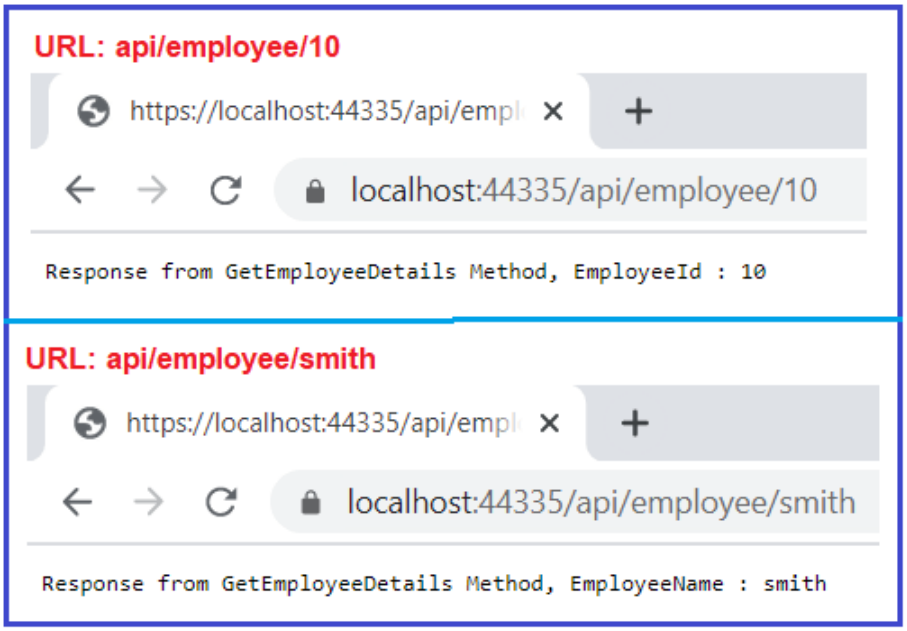
```
        {
            return $"Response from GetEmployeeDetails Method, EmployeeId :
{EmployeeId}";
        }

        [Route("{EmployeeName}")]
        [HttpGet]
        public string GetEmployeeDetails(string EmployeeName)
        {
            return $"Response from GetEmployeeDetails Method, EmployeeName :
{EmployeeName}";
        }
    }
}
```

Expected Behaviour

- /api/employee/10 → Invokes the **integer** version.
- /api/employee/smith → Invokes the **string** version.

With the above changes in place, now run the application and navigate to api/employee/10 and api/employee/smith URLs, and you should get the following output.



Real-time Scenario: Product Management API

Imagine you are designing the backend API for an online shopping platform (like Flipkart or Amazon). Your system contains multiple products, and each **Product** entity can have properties such as:

- **Id** – Unique numeric identifier for each product.

- **Name** – The name of the product.
- **Category** – The category to which the product belongs (e.g., Electronics, Furniture).
- **Rating** – The customer rating of the product on a scale of 1–5.
- **Code** – The internal alphanumeric product code used for identification.
- **SKU (Stock Keeping Unit)** – A standardized pattern-based identifier (e.g., PROD-1001).
- **Price** – The cost of the product, representing its actual market value.

Different types of users (like customers, sellers, or internal tools) might need to:

- Fetch a product by **Numeric ID**
- Filter by **Category Name**
- Search by **Rating**
- Validate **Product Code Format**
- Retrieve products within a **Price Range**
- Match a specific **SKU Pattern**

However, if your API accepts all these filters in routes without validation, it can create ambiguity and errors. This is where **Route Constraints** come in; they tell ASP.NET Core **what kind of data is valid** for each route, ensuring correct request routing and early validation before it reaches your business logic.

So, when exposing product data via URLs, it's essential to ensure that the incoming parameters are **valid**. Invalid data can lead to errors, performance overhead, or even security issues. Route Constraints act as the **First Layer of Validation**, ensuring that only properly formatted and valid requests reach your controller actions.

Example to Understand the Route Constraints in ASP.NET Core Web API

Let us understand this with an example. Please create an Empty API Controller named **ProductController** within the **Controllers** folder, and then copy and paste the following code. The following **Controller** demonstrates **all major ASP.NET Core Route Constraints,** such as int, min, max, range, alpha, minlength, maxlength, length, and regex. The following code is self-explained, so please read the comment lines for a better understanding.

```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductController : ControllerBase
    {
        // In-memory product data (no database)
        private static readonly List<dynamic> Products = new()
        {
```

```
            new { Id = 101, Name = "Laptop",     Category = "Electronics", Rating
= 4, Code = "PRD1234", SKU = "PROD-1001", Price = 55000 },
            new { Id = 102, Name = "Mobile",     Category = "Electronics", Rating
= 5, Code = "PRD5678", SKU = "PROD-1002", Price = 25000 },
            new { Id = 103, Name = "Chair",      Category = "Furniture",   Rating
= 3, Code = "PRD4321", SKU = "PROD-1003", Price = 4500  },
            new { Id = 104, Name = "Table",      Category = "Furniture",   Rating
= 2, Code = "PRD9876", SKU = "PROD-1004", Price = 7800  },
            new { Id = 105, Name = "Mixer",      Category = "Appliances",  Rating
= 5, Code = "PRD6543", SKU = "PROD-1005", Price = 6500  },
            new { Id = 106, Name = "Oven",       Category = "Appliances",  Rating
= 4, Code = "PRD7788", SKU = "PROD-1006", Price = 18500 },
            new { Id = 107, Name = "Headphone",  Category = "Electronics", Rating
= 3, Code = "PRD9988", SKU = "PROD-1007", Price = 4500  },
            new { Id = 108, Name = "Sofa",       Category = "Furniture",   Rating
= 5, Code = "PRD3333", SKU = "PROD-1008", Price = 32000 },
            new { Id = 109, Name = "Fan",        Category = "Appliances",  Rating
= 2, Code = "PRD2222", SKU = "PROD-1009", Price = 2800  },
            new { Id = 110, Name = "Monitor",    Category = "Electronics", Rating
= 4, Code = "PRD1111", SKU = "PROD-1010", Price = 14500 }
        };

        // ------------------------------------------------------------
        // TYPE CONSTRAINT — Fetch Product by Numeric ID
        // Restricts route parameter to integers only.
        // Valid Example:   /api/products/101
        // Invalid Example: /api/products/laptop
        // ------------------------------------------------------------
        [HttpGet("{id:int}")]
        public IActionResult GetProductById(int id)
        {
            var product = Products.FirstOrDefault(p => p.Id == id);
            return product is not null
                ? Ok(product)
                : NotFound($"No product found with ID: {id}");
        }

        // ------------------------------------------------------------
        // RANGE CONSTRAINT — Get Products by Rating (1 to 5)
        // Only accepts numeric values between 1 and 5.
        // Valid Example:   /api/products/rating/4
        // Invalid Example: /api/products/rating/9
        // ------------------------------------------------------------
        [HttpGet("rating/{rating:int:range(1,5)}")]
        public IActionResult GetProductsByRating(int rating)
        {
            var items = Products.Where(p => p.Rating == rating);
```

```
        return items.Any()
            ? Ok(items)
            : NotFound($"No products found with rating: {rating}");
    }

    // ------------------------------------------------------------
    // ALPHA CONSTRAINT — Get Products by Category Name
    // Allows only alphabetic characters (A-Z, a-z).
    // Valid Example:   /api/products/category/electronics
    // Invalid Example: /api/products/category/electronics123
    // ------------------------------------------------------------
    [HttpGet("category/{category:alpha}")]
    public IActionResult GetProductsByCategory(string category)
    {
        var items = Products
            .Where(p => p.Category.ToLower() == category.ToLower());

        return items.Any()
            ? Ok(items)
            : NotFound($"No products found in category: {category}");
    }

    // ------------------------------------------------------------
    // MINLENGTH & MAXLENGTH CONSTRAINTS — Product Code Validation
    // Ensures product code is between 6 and 8 characters long.
    // Valid Example:   /api/products/code/PRD1234
    // Invalid Example: /api/products/code/PRD12
    // ------------------------------------------------------------
    [HttpGet("code/{code:minlength(6):maxlength(8)}")]
    public IActionResult GetProductByCode(string code)
    {
        var product = Products.FirstOrDefault(p => p.Code == code);
        return product is not null
            ? Ok(product)
            : NotFound($"No product found with code: {code}");
    }

    // ------------------------------------------------------------
    // EXACT LENGTH CONSTRAINT — Product Code with Fixed Length
    // Ensures the code must have exactly 7 characters.
    // Valid Example:   /api/products/exact/PRD1234
    // Invalid Example: /api/products/exact/PRD12345
    // ------------------------------------------------------------
    [HttpGet("exact/{code:length(7)}")]
    public IActionResult GetProductByExactLengthCode(string code)
    {
        var product = Products.FirstOrDefault(p => p.Code == code);
```

```
            return product is not null
                ? Ok(product)
                : NotFound($"No product found with exact-length code: {code}");
        }

        // ------------------------------------------------------------
        // REGEX CONSTRAINT — Get Product by SKU Pattern
        // SKU format: starts with "PROD-" followed by exactly 4 digits.
        // Valid Example:   /api/products/sku/PROD-1001
        // Invalid Example: /api/products/sku/prod1001
        // ------------------------------------------------------------
        [HttpGet("sku/{sku:regex(^PROD-[[0-9]]{{4}}$)}")]
        public IActionResult GetProductBySku(string sku)
        {
            // Regex Pattern Explanation:
            // ^        → Start of the string (ensures the SKU begins with the
defined format)
            // PROD-    → The literal text "PROD-" must appear at the beginning
            // [0-9]    → Accepts only digits from 0 to 9
            // {{4}}    → Exactly 4 digits must follow (double braces are needed
to escape in C# route templates)
            // $        → End of the string (ensures nothing comes after the 4
digits)
            //
            // Therefore, the only valid format is:
            // PROD-XXXX → where X represents digits (e.g., PROD-1001, PROD-2045)

            var product = Products.FirstOrDefault(p => p.SKU == sku);
            return product is not null
                ? Ok(product)
                : NotFound($"No product found with SKU: {sku}");
        }

        // ------------------------------------------------------------
        // MIN & MAX CONSTRAINT — Filter Products by Price Range
        // Accepts numeric values where:
        //   minPrice ≥ 100 and maxPrice ≤ 100000.
        // Valid Example:   /api/products/price/5000/to/20000
        // Invalid Example: /api/products/price/50/to/200000
        // ------------------------------------------------------------
        [HttpGet("price/{minPrice:int:min(100)}/to/{maxPrice:int:max(100000)}")]
        public IActionResult GetProductsByPriceRange(int minPrice, int maxPrice)
        {
            // Validate logical range — avoids inverted ranges
            if (minPrice > maxPrice)
                return BadRequest("Minimum price cannot be greater than maximum
price.");
```

```
            // Filter products whose price lies within the given range
            var items = Products.Where(p => p.Price >= minPrice && p.Price <=
maxPrice);

            // Return appropriate response
            return items.Any()
                ? Ok(items)
                : NotFound($"No products found within the price range ₹{minPrice}
- ₹{maxPrice}");
        }

        // ----------------------------------------------------------------
        // MULTIPLE CONSTRAINTS — Category + Rating Combination
        // Demonstrates combining alpha and range constraints.
        // Valid Example:   /api/products/filter/electronics/4
        // Invalid Example: /api/products/filter/electronics/8
        // ----------------------------------------------------------------
        [HttpGet("filter/{category:alpha}/{rating:int:range(1,5)}")]
        public IActionResult GetProductsByCategoryAndRating(string category, int
rating)
        {
            var items = Products.Where(p =>
                p.Category.ToLower() == category.ToLower() && p.Rating ==
rating);

            return items.Any()
                ? Ok(items)
                : NotFound($"No {category} products found with rating {rating}");
        }
    }
}
```

Now, run the application and test the endpoints with both valid and invalid route data; it should work.

Custom Route Constraint in ASP.NET Core Web API

A **Custom Route Constraint** is a user-defined rule that extends the built-in routing system of ASP.NET Core to handle **Special Validation Logic** that cannot be achieved using standard Route Constraints (like int, alpha, range, etc.). It allows developers to implement their own logic to determine whether a route parameter is valid before the request reaches the controller action.

Custom constraints are a class that implements the IRouteConstraint interface, giving us complete control over how route matching is performed.

Syntax: class CustomConstraint : IRouteConstraint

Why Do We Need a Custom Route Constraint?

While built-in constraints handle most of the common scenarios (like numeric or alphabetic validation), real-world business applications often require **domain-specific validation rules**. For example:

- A **Product Code** must start with “PRD” and contain exactly four digits.
- A **Category Name** must exist in a predefined list of allowed categories. That means allow only specific product categories such as **Electronics, Furniture, or Appliances**.
- A **Country Code** must match ISO standards like “IN”, “US”, “UK”.
- A product’s SKU starts with a brand prefix or location code (e.g., IND- for India).

Such validations cannot be handled using basic constraints or even simple regex sometimes, that’s when a **Custom Route Constraint** becomes necessary. A **Custom Route Constraint** helps you enforce such rules directly at the routing level, improving **API Performance, Readability, and Security**, before executing the action method.

Real-Time Scenario: Product Management API

In our **Product Management API**, we have various product categories such as Electronics, Furniture, and Appliances. Now, imagine we want to restrict our API to accept only these valid categories, rejecting anything else (e.g., /api/products/category/toys should not be processed).

Since the built-in alpha constraint only validates alphabetic input but not specific values, we need a **Custom Route Constraint** that validates **Allowed Categories only**.

Step 1: Create a Custom Route Constraint

First, create a folder named **Constraints** at the project root directory. Then, add a class file named **AllowedCategoriesConstraint.cs** within the **Constraints** folder, and copy-paste the following code. The following is a Custom route constraint to validate allowed product categories.

```
namespace RoutingInASPNETCoreWebAPI.Constraints
{
    // Custom route constraint that validates whether
    // a given category parameter matches one of the predefined valid categories.
    public class AllowedCategoriesConstraint : IRouteConstraint
    {
        // ------------------------------------------------------------------
        // STEP 1: Define allowed values for the 'category' route parameter.
        // ------------------------------------------------------------------
        // Only the following categories are considered valid.
```

```
        // Any other category (e.g., "toys" or "clothing") will cause
        // the route to fail, preventing the controller action from executing.
        private static readonly string[] AllowedCategories =
            { "electronics", "furniture", "appliances", "stationery" };

        // ------------------------------------------------------------
        // STEP 2: Implement the Match() method of IRouteConstraint.
        // ------------------------------------------------------------
        // The Match() method determines whether a route parameter value
        // satisfies the constraint logic.
        //
        // Parameters:
        //  - httpContext: Provides access to the current HTTP context.
        //  - route: Represents the current route being evaluated by the routing
system..
        //  - parameterName: The name of the route parameter being checked (e.g.,
"category").
        //  - values: Dictionary of all route parameters and their values.
        //  - routeDirection: indicates whether the check is being performed
        //                    during URL matching (IncomingRequest)
        //                    or URL generation (OutgoingResponse).
        //
        // Returns:
        //  - true  → if the parameter value passes validation (route matches)
        //  - false → if the parameter value fails validation (route ignored)
        // ------------------------------------------------------------
        public bool Match(HttpContext? httpContext, IRouter? route, string
parameterName,
                          RouteValueDictionary values, RouteDirection
routeDirection)
        {
            // ------------------------------------------------------------
            // STEP 3: Safely extract the parameter value from the route.
            // ------------------------------------------------------------
            // If the parameter is missing from the route (null or undefined),
            // the constraint automatically fails.
            if (!values.TryGetValue(parameterName, out var rawValue))
                return false;

            // Convert the parameter value to string.
            var category = rawValue?.ToString();

            // ------------------------------------------------------------
            // STEP 4: Validate the category against the allowed list.
            // ------------------------------------------------------------
            //  - Ensure the category is not null or empty.
            //  - Use Array.Exists() to check if it matches any predefined value.
```

```
            //  - Comparison is done using OrdinalIgnoreCase for efficiency and
accuracy.
            return !string.IsNullOrEmpty(category) &&
                   Array.Exists(AllowedCategories, c => c.Equals(category,
StringComparison.OrdinalIgnoreCase));
        }
    }
}
```

Step 2: Register the Custom Constraint in the Program.cs

Then, we must register our custom route constraint so that ASP.NET Core recognizes it in route templates. So, please modify the Program class as follows:

```
using RoutingInASPNETCoreWebAPI.Constraints;
namespace RoutingInASPNETCoreWebAPI
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.
            builder.Services.AddControllers();
            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();

            // ------------------------------------------------------------
            // Register Custom Route Constraint
            // ------------------------------------------------------------
            // Here, we tell ASP.NET Core's routing system that we have created
            // a new custom route constraint named "allowedCategories".
            //
            // The key ("allowedCategories") is the name we will use in route
templates,
            // e.g.  [HttpGet("category/{category:allowedCategories}")]
            //
            // The value (typeof(AllowedCategoriesConstraint)) specifies the C#
class
            // that implements IRouteConstraint and contains the logic
            // defining which categories are valid.
            //
            // Once registered, the routing middleware will automatically call
            // the Match() method of AllowedCategoriesConstraint
            // whenever a route contains {parameter:allowedCategories}.
            builder.Services.Configure<RouteOptions>(options =>
```

```
            {
                options.ConstraintMap.Add("allowedCategories",
typeof(AllowedCategoriesConstraint));
            });

            var app = builder.Build();

            // Configure the HTTP request pipeline.
            if (app.Environment.IsDevelopment())
            {
                app.UseSwagger();
                app.UseSwaggerUI();
            }

            app.UseHttpsRedirection();

            app.UseAuthorization();

            app.MapControllers();

            app.Run();
        }
    }
}
```

Step 3: Use the Custom Constraint in the Controller

Now, apply it inside your ProductController route just like any other built-in constraint. So, please modify the following GetProductsByCategory method within the ProductController.

```
// ------------------------------------------------------------
// CUSTOM ROUTE CONSTRAINT – Validate Allowed Product Categories
// This ensures that only predefined categories are accepted.
// Valid Example:   /api/products/category/electronics
// Invalid Example: /api/products/category/toys
// ------------------------------------------------------------
[HttpGet("category/{category:allowedCategories}")]
public IActionResult GetProductsByAllowedCategory(string category)
{
    var items = Products.Where(p => p.Category.Equals(category,
StringComparison.OrdinalIgnoreCase));

    return items.Any()
        ? Ok(items)
        : NotFound($"No products found under the '{category}' category.");
}
```

Explanation

- The **AllowedCategoriesConstraint** class validates that only specific categories (Electronics, Furniture, Appliances, and Stationery) are accepted in the route.
- If the user tries /api/products/category/toys, the request **Never Reaches the Controller** because the routing system rejects it.
- This ensures invalid requests are filtered out early, improving **Performance and Security**.

Can we apply Route Constraints to Query String Parameters?

No, Route Constraints in ASP.NET Core **cannot be applied to query string parameters**. They only apply to **Route Parameters,** i.e., the portions of the URL **Path** that are part of the routing template.

Query String Parameters (like **?id=5** or **?category=electronics**) are **not part of the route template**; they are processed **after routing has already occurred.**

For example, a request such as: **GET /api/products?id=5** does not participate in route matching. By the time ASP.NET Core reads id=5, routing has already selected which controller and action to run.

To validate Query String Parameters, use **Model Validation**, **Data Annotations**, or **Manual Validation Logic** inside your controller action.

When to Use Custom Route Constraints in ASP.NET Core Web API

Use them when:

- Built-in constraints (like int, alpha, regex) are not enough.
- You need **Domain-Specific Rules**, e.g., product category, SKU prefix, or region code.
- You want validation to happen **before** your controller logic runs.

Route constraints in ASP.NET Core Web API add precision, safety, and clarity to your routing mechanism. By enforcing specific types, ranges, or formats at the route level, they prevent invalid requests from entering the pipeline, resulting in cleaner, faster, and more secure APIs. In short, route constraints turn routing into your first layer of validation.

In the next article, I will discuss **How Routing Works in ASP.NET Core Web API** Applications with Examples. In this article, explain Route Constraints in ASP.NET Core Web API with examples. And I hope you enjoy this article.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.